

## Tarea 2

### *Parsing e Intérpretes*

Para la resolución de la tarea recuerde que

- Toda función debe estar acompañada de su firma, una breve descripción coloquial (en `T2.rkt`) y un conjunto significativo de tests (en `test.rkt`).
- Todo datatype definido por el usuario (via `deftype`) debe estar acompañado de una breve descripción coloquial y de la gramática BNF que lo genera.

Si la función o el datatype no cumple con estas reglas, será ignorado.

#### Ejercicio 1

22 Pt

Se define un lenguaje aritmético cuyos programas denotan polinomios, en lugar de números. La sintaxis abstracta del lenguaje está dada por la siguiente gramática BNF:

```
#|  
<poly> ::= (poly (Listof <number>))  
         | (id <id>)  
         | (add <poly> <poly>)  
         | (mul <poly> <poly>)  
         | (if0 <poly> <poly> <poly>)  
         | (with <id> <poly> <poly>)  
|#
```

Un polinomio está representado por una lista de números, en donde cada posición contiene el coeficiente correspondiente a ese grado. Por ejemplo, `(poly '(4 0 5))` representa al polinomio  $4 + 5x^2$ .

Dentro de las construcciones del lenguaje están los identificadores, la suma y el producto de polinomios, un condicional que compara la guarda con el polinomio nulo (`(poly '(0))`) y la definición local que asocia un identificador con un polinomio para que pueda ser referenciado en el cuerpo de la misma.

(a) [6 Pt] Defina el tipo de datos recursivo `Poly` que genera los AST de los programas del lenguaje.

(b) [6 Pt] Defina la sintaxis concreta del lenguaje y escriba la función

```
;; parse :: <s-poly> -> Poly
```

que devuelve los programas escritos en sintaxis abstracta. Para ello, considere los siguientes ejemplos:

```

>>> (parse 0)
(poly '(0))
>>> (parse '(3 1 0 0))
(poly '(3 1))
>>> (parse 'x)
(id 'x)
>>> (parse '(+ (3 5 7) (1 2)))
(add (poly '(3 5 7)) (poly '(1 2)))
>>> (parse '(* (2 1) (1 2)))
(mul (poly '(2 1)) (poly '(1 2)))
>>> (parse '(if0 (+ (3 5) (-3 2)) 1 0))
(if0 (add (poly '(3 5)) (poly '(-3 2))) (poly '(1)) (poly '(0)))
>>> (parse '(with x (1 2) (* 2 x)))
(with 'x (poly '(1 2)) (mul (poly '(2)) (id 'x)))

```

- (c) [10 Pt] Sea la estructura `Env` que modela un ambiente en donde se tienen las relaciones entre los identificadores y sus valores en forma de polinomios puros, es decir, sin operaciones y únicamente formado por el constructor `poly`. Defina la función

```
;; reduce :: Poly Env -> Poly
```

que devuelve un polinomio puro.

```

>>> (reduce (poly '(3 1 0)) empty-env)
(poly '(3 1))
>>> (reduce (id 'x) empty-env)
reduce: variable x is not defined
>>> (reduce (id 'x) (extend-env x (poly '(1)) empty-env))
(poly '(1))
>>> (reduce (add (poly '(3 5 7)) (poly '(1 2))) empty-env)
(poly '(4 7 7))

```

## Ejercicio 2

38 Pt

El archivo `T2.rkt` contiene una definición inicial del tipo de datos recursivo `Expr`, basado en el lenguaje visto en clases.

- (a) [6 Pt] Extienda `Expr` (y su gramática BNF) con operadores números reales e imaginarios. Además, agregue una forma de introducir varias definiciones locales en una sola expresión, es decir, una forma de `with` con varias definiciones. Para ello, utilice los siguientes ejemplos como guía (no es necesario implementar la función `parser` aún):

- Números reales e imaginarios:

```

>>> (parser '1)
(real 1)
>>> (parser '(1 i))
(imaginary 1)
>>> (parser '(+ 2 (1 i)))
(addc (real 2) (imaginary 1))

```

- Identificadores y definiciones locales:

```
>>> (parser '(with [(x 1) (y 1)] (+ x y)))
(withc (list (cons 'x (real 1)) (cons 'y (real 1))) (addc (idc 'x) (idc 'y)))
```

- (b) [6 Pt] Implemente la función `parser` siguiendo los ejemplos anteriores. Además, considere que la expresión `with` siempre debe recibir al menos una definición y en caso contrario se debe lanzar una excepción:

```
>>> (parser '(with [] 1))
parser: *with* expects at least one definition
```

- (c) [6 Pt] En el archivo `base`, se encuentra una definición de valores `CValue` para el lenguaje. Dado que este trabaja con números reales e imaginarios, naturalmente los valores son los números complejos. Implemente la función `from-CValue` que convierte un `CValue` a un elemento `Expr`. Además, implemente las funciones `cmplx+`, `cmplx-` y `cmplx0`. Las primeras dos funciones suman y restan, respectivamente, dos valores del lenguaje. La última función retorna `#t` si el número complejo es 0.

```
>>> (cmplx+ (compV 1 2) (compV 3 4))
(compV 4 6)
>>> (cmplx- (compV 1 2) (compV 3 4))
(compV -2 -2)
>>> (cmplx0? (compV 0 1))
#f
```

- (d) [10 Pt] Implemente la función `subst` que realiza la substitución de una expresión por un identificador. Note que una expresión `with` introduce identificadores que pueden ser utilizados en próximas definiciones, permitiendo que expresiones como la siguiente sean válidas:

```
(with [(x 2) (y (+ x 1))] (+ x y))
```

La definición de `y` ocupa `x`.

Su función de substitución debe tomar esto en cuenta y evitar substituir variables que han sido "oscurecidas" por otra definición local. Por ejemplo:

```
;; Este ejemplo no tiene "shadowing"
>>> (subst (parser '(with [(x 2) (y z)] (+ x z))) 'z (real 1))
(withc (list (cons 'x (real 2)) (cons 'y (real 1))) (addc (idc 'x) (real 1)))
;; En este ejemplo si ocurre "shadowing" -> no se realiza la substitucion
>>> (subst (parser '(with [(x 2) (y x)] (+ x x))) 'x (real 1))
(withc (list (cons 'x (real 2)) (cons 'y (idc 'x))) (addc (idc 'x) (idc 'x)))
```

*Hint:* Le puede ser útil definir una función auxiliar que realice una substitución sobre una lista de definiciones.

- (e) [10 Pt] Implemente la función `interp` que reduce una expresión (`Expr`) en un valor del lenguaje (`CValue`). Si se encuentra una variable libre, arroje un error con el mensaje: *"Free occurrence of a variable"*.